

BLOQUE

3

Backend seguro y API documentada

FASE CONSTRUCTION • CONSTRUCCIÓN COLABORATIVA (MOB CONSTRUCTION)

CONEXIÓN AI-DLC

La IA propone endpoints y esquema; el equipo los revisa, sobre todo en seguridad.

EVIDENCIA DEL EQUIPO

Backend del MVP + documentación Swagger + informe OWASP + matriz de trazabilidad.

EVALÚA

Matriz 2 (backend, 50 pts) — receptivo → estratégico.

Flask y Blueprints

```
# backend/app/tareas/routes.py
from flask import Blueprint, jsonify, request

tareas_bp = Blueprint("tareas", __name__, url_prefix="/api/tareas")

@tareas_bp.get("")
def listar():
    return jsonify(servicio.listar_del_usuario(g.user_id)), 200
```

- Un **Blueprint por recurso**. Nada de un `app.py` de 800 líneas.

Diseñar la base de datos



3+ entidades, relaciones con claves foráneas, e índices en las columnas por las que se filtra.

Autenticación y autorización con JWT

- **AuthN** = quién sos (login). **AuthZ** = qué podés hacer (rol).
- El **JWT** es un token firmado con `claims` (id, rol, expiración).
- Expiración corta (~15 min) + **refresh token** para renovar.

```
                                @requiere_auth(rol="admin")
def eliminar_usuario(id):
    # solo llega aquí si el token es válido y el rol es admin
    ...
```

Validar entrada y consultar sin inyección

```
# MAL – concatenar construye la inyección SQL  
db.execute(f"SELECT * FROM tarea WHERE id = {id}")  
  
# BIEN – consulta parametrizada  
db.execute("SELECT * FROM tarea WHERE id = %s", (id,))
```

- Nunca confiar en el cliente: validar **tipo, rango y formato** en el backend.
- Sanitizar lo que se muestra para evitar XSS almacenado.

Checklist OWASP para la API

- **Control de acceso roto** — ¿cada endpoint verifica permiso, no solo sesión?
- **Inyección** — ¿todas las consultas parametrizadas?
- **Configuración insegura** — ¿CORS cerrado? ¿sin modo debug en producción?
- **Secretos** — ¿todas las credenciales en `.env`, ninguna en el repo?
- **Exposición de datos** — ¿la API devuelve solo los campos necesarios?

Variables de entorno y CORS

```
# .env (en .gitignore – nunca al repositorio)  
DATABASE_URL=postgresql://...  
JWT_SECRET=...  
CORS_ORIGINS=https://mi-proyecto.pages.dev
```

```
# CORS: solo los orígenes autorizados, nunca "*" en producción  
CORS(app, origins=os.environ["CORS_ORIGINS"].split(","))
```

Documentar la API con Swagger

- Swagger / OpenAPI describe cada endpoint: **método, parámetros, cuerpo, respuestas** y códigos.
- Se genera una página interactiva para probar la API sin escribir código.
- La organización cliente puede leerla y entender qué hace el sistema.

IDEA CLAVE

Una API sin documentar no cumple estándares de calidad , aunque funcione. Documentar es parte del entregable, no un extra.

Revisar, sobre todo, la seguridad

- La IA genera código inseguro con frecuencia: SQL concatenado, sin validación, secretos en el código, CORS abierto.
- Al aprobar un endpoint que propuso la IA, correr el checklist OWASP **sobre ese endpoint**.
- Registrar en la bitácora qué generó y qué se corrigió.

IDEA CLAVE

Un endpoint que la IA escribió y nadie auditó es una puerta abierta que nadie vigila .

Bloque 3 — cómo cierra

CONEXIÓN AI-DLC

Fase Construction : la IA propone endpoints y esquema; el equipo los revisa en seguridad antes de aprobar.

EVIDENCIA

Backend del MVP + documentación
Swagger + informe OWASP + matriz de trazabilidad (requisito → endpoint → tabla) + bitácora.

EVALÚA

Matriz 2 (backend, 50 pts) · receptivo → estratégico. El estratégico exige API documentada, versionada y seguridad auditada.